

Computer Vision Practical Report:

WATER MARKING

1. AIM

To insert a text or logo watermark onto an image using alpha blending (`cv2.addWeighted()`) in OpenCV, display the original and watermarked images, and save the result to disk.

2. PROCEDURE

- Load the primary image and the watermark (logo/text mask) using `cv2.imread()`.
- Resize the watermark image or define text position parameters (`cv2.putText()`).
- Extract the Region of Interest (ROI) from the primary image matching the watermark dimensions.
- Blend the watermark into the ROI using `cv2.addWeighted()` to control transparency.
- Replace the original ROI with the blended result, display side-by-side using Matplotlib, and save using `cv2.imwrite()`.

3. PROGRAM CODE

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Define image path
image_path = 'E26-IMG.jpg'

# Check if file exists
if not os.path.exists(image_path):
    print(f"Error: Could not find '{image_path}'. Make sure the image is saved in the same directory.")
else:
    # Load input image and convert to RGB for Matplotlib
    image = cv2.imread(image_path)
```

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# 2. Create an overlay copy for transparent watermarking
overlay = image_rgb.copy()

# 3. Watermark Properties
text = "CONFIDENTIAL"
font = cv2.FONT_HERSHEY_SIMPLEX
font_scale = 1.5
color = (255, 255, 255) # White text (RGB)
thickness = 3

# Calculate position to center the text
h, w, _ = image_rgb.shape
text_size, _ = cv2.getTextSize(text, font, font_scale, thickness)
text_w, text_h = text_size
pos_x = (w - text_w) // 2
pos_y = (h + text_h) // 2

# Draw text on the overlay
cv2.putText(overlay, text, (pos_x, pos_y), font, font_scale, color, thickness, cv2.LINE_AA)

# 4. Blend overlay with original image to apply transparency
alpha = 0.4 # Transparency factor (0.0 = completely transparent, 1.0 = fully opaque)
watermarked_rgb = cv2.addWeighted(overlay, alpha, image_rgb, 1 - alpha, 0)

# 5. Display Input and Watermarked Output on a SINGLE PAGE
plt.figure(figsize=(10, 5))

# Original Image
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title('Original Image')
plt.axis('off')
```

```
# Watermarked Image
plt.subplot(1, 2, 2)
plt.imshow(watermarked_rgb)
plt.title("Watermarked Image")
plt.axis('off')

plt.tight_layout()
plt.show()
```

4. OUTPUT

Original Image



Watermarked Image



5. RESULT

The Python script executed successfully. The input image was loaded.